Adaptive Cognitive Load Prediction System for Online Learning Using Behavioral Patterns

AWS EC2 Deployment Report

Group ID R26-SE-004

Bachelor of Science Honours in Information Technology

Specialization in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

TABLE OF CONTENT

1. Introduction 3
2. AWS EC2 Instance and Elastic IP Configuration 3
2.1 EC2 Instance Selection and Resource Optimization 3
2.2 Elastic IP Configuration 4
3. Docker Compose Deployment and Container Networking 5
4. API Gateway, NGINX, and Port Mapping 7
5. MySQL Database Initialization and Persistent Storage 8
6. Deployment Verification and Testing Status 9
7. Deployment Challenges and Resolutions 10
8. Security and Reliability Considerations 11

TABLE OF FIGURES

Figure 1: AWS EC2 instance summary showing the running instance, public IPv4 address, Elastic IP address, and instance type. 4
Figure 2 AWS EC2 instance details displaying the Amazon Linux configuration and launch specifications 5
Figure 3 Terminal output showing sudo docker-compose ps with all active microservices containers running and their corresponding port mappings. 5
Figure 4 AWS EC2 Storage and EBS block device configuration (50 GiB gp3 root volume) highlighting storage metrics and I/O monitoring. 8

LIST OF TABLES

Table 1:Docker service port mappings showing the host and container ports configured for each microservice. 7
Table 2: Deployment verification and testing status of the LUMORA application components and workflows. 9
Table 3: Deployment challenges encountered and their corresponding resolutions during the AWS EC2 deployment of the LUMORA application. 10

1. Introduction

This report reviews the deployment architecture and configuration of the LUMORA application. It is an AI-driven online learning platform developed using microservices architecture. The application consists of a frontend, an API Gateway, multiple backend and AI services, and a MySQL database.

The application has been deployed to an Amazon EC2 Linux instance using Docker and Docker Compose. Docker images package individual services and their dependencies, while Docker Compose manages service configuration, container creation, port mapping, networking, and startup behavior. The deployment uses a custom Docker bridge network to enable internal communication between microservices through service-name-based DNS resolution. A public IPv4 address, intended to be the Elastic IP associated with the EC2 instance, is used to access the application externally.

The deployment configuration and functionality of the LUMORA application were evaluated by examining the AWS EC2 environment, Docker Compose services, container networking, and application endpoints. The deployment process focused on ensuring proper communication between microservices, database connectivity, and successful operation of the application's main functionalities.

2. AWS EC2 Instance and Elastic IP Configuration

Amazon EC2 provides the Linux based computing environment used to deploy and operate the Dockerized application. The instance supplies the CPU, memory, storage, and network connectivity required to run the frontend, API Gateway, backend microservices, AI services, and MySQL database.

2.1 EC2 Instance Selection and Resource Optimization

During the initial deployment, a t4g.nano instance was selected to host the application. However, its limited computing resources were insufficient to run the application's multiple Docker containers reliably. The instance was subsequently upgraded to a t3.small instance, but the available resources were still insufficient to support the complete workload effectively.

To improve deployment stability, additional storage was allocated to the EC2 environment, bringing the available storage to 50 GB. Memory utilization was also addressed by configuring additional swap space, allowing the operating system to use disk space as supplementary virtual memory when physical RAM becomes limited. These adjustments helped the instance accommodate the resource requirements of the containerized application. Although swap space can reduce memory related failures, it is slower than physical RAM and does not replace the need for sufficient memory. Therefore, regular monitoring is important when running multiple

microservices on a single EC2 instance. This includes monitoring CPU utilization, memory consumption, disk usage, and container health.

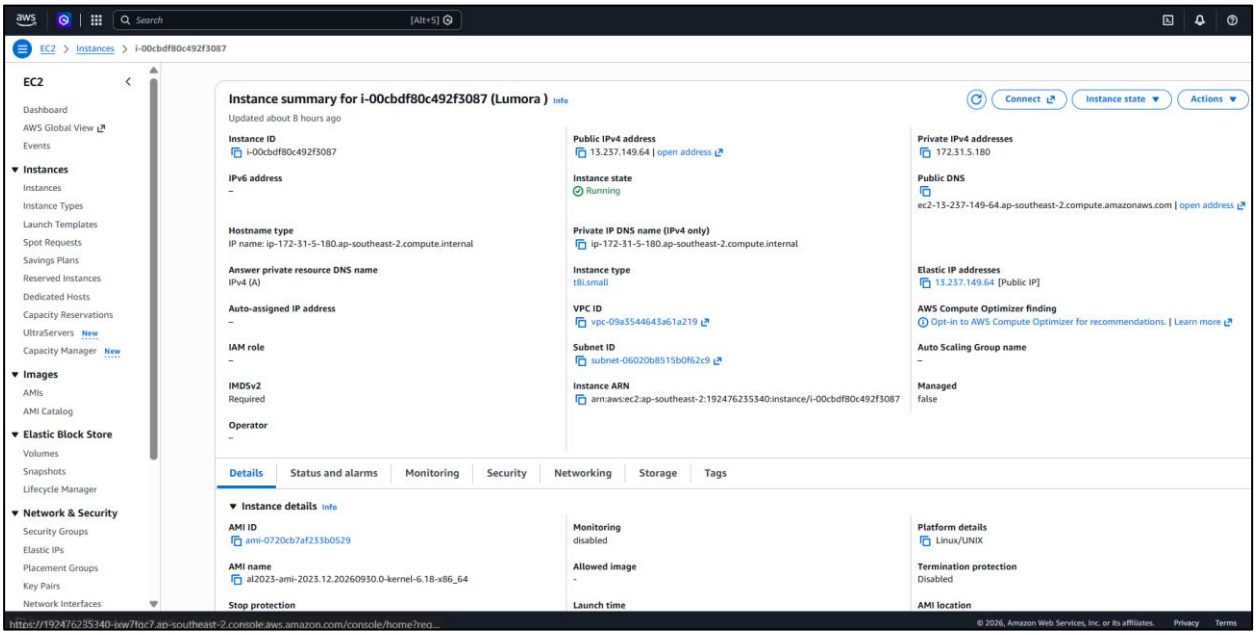


Figure 1: AWS EC2 instance summary showing the running instance, public IPv4 address, Elastic IP address, and instance type.

2.2 Elastic IP Configuration

An Elastic IP is a static public IPv4 address that can be associated with an AWS resource, such as an EC2 instance or network interface. It provides a stable public address while the association remains in place. This allows external clients to access the application without relying on a changing automatically assigned public IPv4 address.

The intended external access flow is as follows:

1. A user sends an HTTP or HTTPS request to the public IP address or domain name.
2. AWS networking and the EC2 Security Group determine whether the incoming traffic is permitted.
3. Docker forwards traffic arriving on a published host port to the corresponding container port.
4. The frontend or API Gateway processes the request and forwards it to the appropriate application service.

The frontend build configuration contains the following API Gateway URL: http://3.27.27.96:4000

This address must match the Elastic IP currently associated with the EC2 instance. Since other public IP addresses were referenced during earlier deployment stages, the configured address should be verified against the current AWS configuration and the URL embedded in the deployed frontend build.

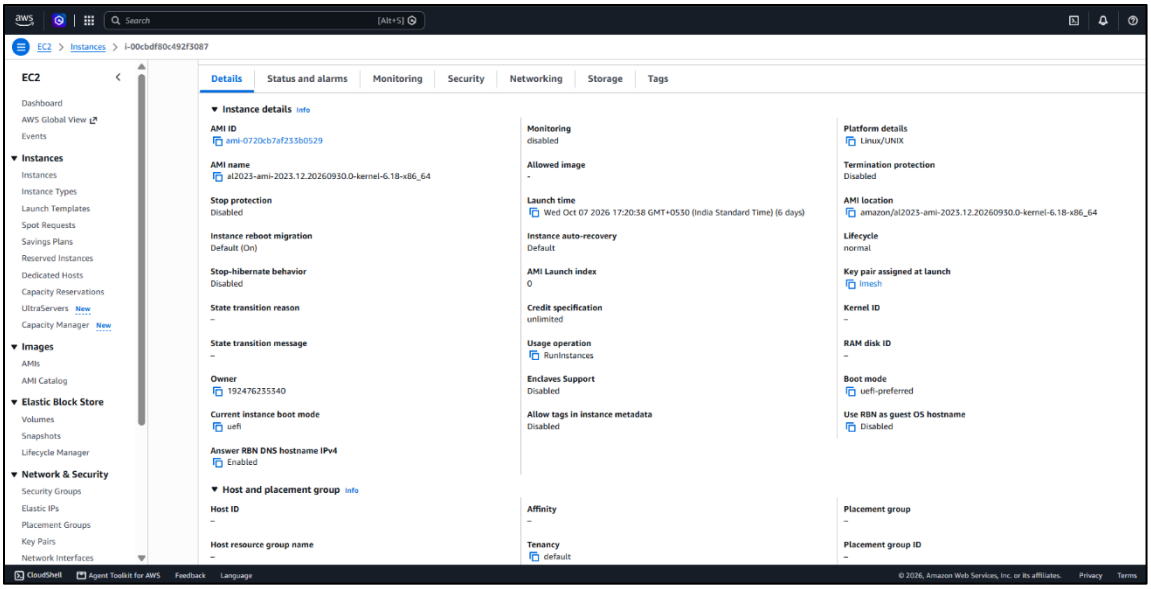


Figure 2 AWS EC2 instance details displaying the Amazon Linux configuration and launch specifications.

3. Docker Compose Deployment and Container Networking

Docker Compose was used to build, configure, and manage the application's microservices through the centralized ***docker-compose.yaml*** file. The configuration defines each service's Docker build context, environment variables, port mappings, restart policies, and network settings.

```
[ec2-user@ip-172-31-5-180 recommendation_ai]$ sudo docker-compose ps
NAME                              IMAGE                             COMMAND                  SERVICE              CREATED             STATUS              PORTS
rp_aws_host-api-gateway-1         rp_aws_host-api-gateway           "docker-entrypoint.s…"   api-gateway          5 days ago          Up 5 days           0.0.0.0:4000->4000/tcp, :::4000->4000/tcp
rp_aws_host-backend-1             rp_aws_host-backend               "docker-entrypoint.s…"   backend              40 hours ago        Up 40 hours         0.0.0.0:5001->5001/tcp, :::5001->5001/tcp
rp_aws_host-cognitive-backend-1   rp_aws_host-cognitive-backend     "uvicorn main:app --…"   cognitive-backend    40 hours ago        Up 40 hours         0.0.0.0:8003->8003/tcp, :::8003->8003/tcp
rp_aws_host-cognitive-load-api-1  rp_aws_host-cognitive-load-api   "uvicorn app.main:ap…"   cognitive-load-api   7 hours ago         Up 7 hours          0.0.0.0:8021->8021/tcp, :::8021->8021/tcp
rp_aws_host-cognitive-style-ai-1  rp_aws_host-cognitive-style-ai   "uvicorn main:app --…"   cognitive-style-ai   5 hours ago         Up 5 hours          0.0.0.0:8112->8112/tcp, :::8112->8112/tcp
rp_aws_host-deepseek-service-1    rp_aws_host-deepseek-service     "docker-entrypoint.s…"   deepseek-service     5 days ago          Up 5 days           0.0.0.0:5004->5004/tcp, :::5004->5004/tcp
rp_aws_host-exam-service-1        rp_aws_host-exam-service         "docker-entrypoint.s…"   exam-service         6 days ago          Up 6 days           0.0.0.0:8120->8120/tcp, :::8120->8120/tcp
rp_aws_host-frontend-1            rp_aws_host-frontend             "/docker-entrypoint.…"   frontend             5 days ago          Up 5 days           0.0.0.0:80->80/tcp, :::80->80/tcp
rp_aws_host-gpt-service-1         rp_aws_host-gpt-service          "docker-entrypoint.s…"   gpt-service          5 days ago          Up 5 days           0.0.0.0:5002->5002/tcp, :::5002->5002/tcp
rp_aws_host-lime-ai-1             rp_aws_host-lime-ai              "uvicorn main:app --…"   lime-ai              11 minutes ago      Up 11 minutes       0.0.0.0:8110->8110/tcp, :::8110->8110/tcp
rp_aws_host-mysql-db-1            mysql:8.0                        "docker-entrypoint.s…"   mysql-db             10 hours ago        Up 10 hours         0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp
rp_aws_host-recommendation-ai-1   rp_aws_host-recommendation-ai    "docker-entrypoint.s…"   recommendation-ai    5 minutes ago       Up 5 minutes        0.0.0.0:5003->5003/tcp, :::5003->5003/tcp
rp_aws_host-resource-upload-1     rp_aws_host-resource-upload      "docker-entrypoint.s…"   resource-upload      5 days ago          Up 5 days           0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
rp_aws_host-sharp-ai-1            rp_aws_host-sharp-ai             "uvicorn main:app --…"   sharp-ai             16 minutes ago      Up 16 minutes       0.0.0.0:8111->8111/tcp, :::8111->8111/tcp
[ec2-user@ip-172-31-5-180 recommendation_ai]$
```

Figure 3 Terminal output showing sudo docker-compose ps with all active microservices containers running and their corresponding port mappings.

The deployment includes,

- Frontend
- API Gateway
- backend,
- resource upload service,
- GPT service,
- recommendation service,
- DeepSeek service
- cognitive style backend,
- cognitive-load API,
- LIME AI
- SHAP AI,
- cognitive-style AI,
- exam service
- MySQL database

Docker Compose builds the required images and starts the containers within the AWS EC2 environment.

All application containers are connected through a custom Docker bridge network configured as app-network, which appears as ***rp_aws_host_app-network*** in the deployed environment. Internal communication is configured using Docker Compose service names and their respective ports. For example, the API Gateway communicates with the backend using ***http://backend:5001***, while database connections use the MySQL service hostname ***mysql-db***.

During deployment, some services experienced DNS resolution issues because the configured service names did not match those defined in ***docker-compose.yaml***. These issues were resolved by correcting the service names and replacing internal ***localhost*** URLs with the appropriate Docker service names.

The following commands were used to manage and troubleshoot the deployment:

- ***sudo docker compose build*** — builds images for services configured with ***build:.***
- ***sudo docker compose up -d*** — starts the configured containers in detached mode.
- ***sudo docker p***s — verifies running containers and published ports.
- ***sudo docker images*** — lists locally available Docker images.
- ***sudo docker compose logs --tail=200 SERVICE_NAME*** — checks individual service logs during troubleshooting.

After correcting the configuration, the Docker containers and internal service communication were verified. The frontend, API Gateway, backend microservices, AI services, and MySQL database were able to operate within the configured Docker environment.

4. API Gateway, NGINX, and Port Mapping

The API Gateway is used as the central entry point for routing requests to the appropriate backend and AI microservices. NGINX is configured in the frontend container to serve the frontend application and proxy API requests to the API Gateway.

The following port mappings are configured for the application's Docker services:

Table 1: Docker service port mappings showing the host and container ports configured for each microservice.

Service	Host Port	Container Port
Frontend (NGINX)	80	80
API Gateway	4000	4000
Backend	5001	5001
Resource Upload	5000	5000
GPT Service	5002	5002
Recommendation AI	5003	5003
DeepSeek Service	5004	5004
Cognitive Backend	8003	8003
Cognitive Load API	8021	8021
LIME AI	8110	8110
SHAP AI	8111	8111
Cognitive Style AI	8112	8112
Exam Service	8120	8120
MySQL	3306	3306

These mappings represent the configured Docker Compose settings. The frontend is accessed through port 80, while the API Gateway uses port 4000. The remaining services use their configured ports for application communication.

NGINX handles frontend requests and forwards requests under ***/api/*** to the API Gateway according to its proxy configuration. The API Gateway then routes requests to the relevant services using their Docker service names and internal ports.

During deployment, issues related to API routing, incorrect service URLs, and NGINX proxy configuration were investigated and corrected. The frontend gateway URL and internal service addresses were updated to support communication between the deployed components. After these corrections, the frontend, API Gateway, and relevant application workflows were tested to verify that requests reached the appropriate services successfully.

5. MySQL Database Initialization and Persistent Storage

The MySQL service uses the official MySQL 8.0 image to provide relational database storage for the application. Two SQL initialization scripts are mounted into the container to initialize the required databases and database structures during the initial setup of an empty MySQL data directory.

The deployment uses 50 GB of storage attached to the AWS EC2 instance to accommodate the application files, Docker images, container data, and database storage. MySQL database files are maintained on a separate persistent volume, allowing database storage to remain independent of the MySQL container lifecycle.

This arrangement allows the database container to be stopped, restarted, or recreated without losing existing database files. However, the persistent volume must be retained and correctly attached to ensure data preservation. The available storage also provides space for application operations and database growth. To improve reliability, storage utilization should be monitored, and regular database backups should be maintained separately from the EC2 instance. Access to MySQL should also be restricted to authorized application components and administrative systems.

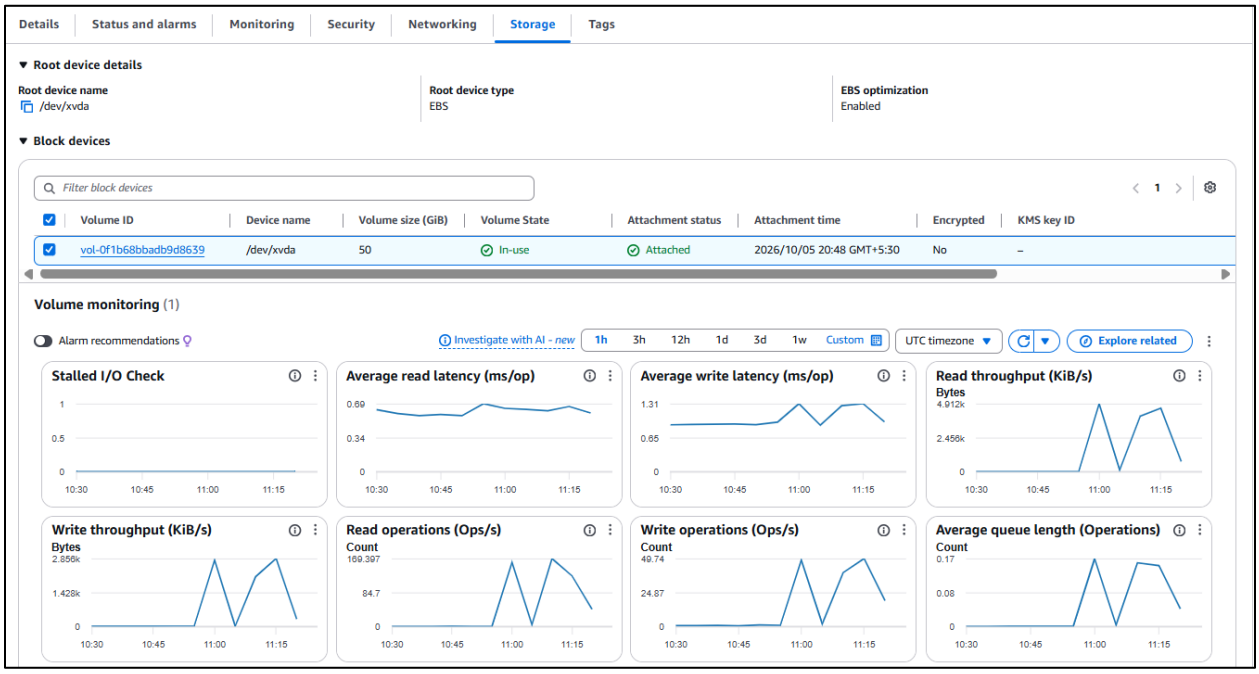


Figure 4 AWS EC2 Storage and EBS block device configuration (50 GiB gp3 root volume) highlighting storage metrics and I/O monitoring.

6. Deployment Verification and Testing Status

After deploying the application on AWS EC2, verification and testing were performed to confirm infrastructure availability, communication between microservices, database connectivity, and the functionality of the main application workflows.

Table 2: Deployment verification and testing status of the LUMORA application components and workflows.

Component or workflow	Verification status
AWS EC2 deployment	Successfully deployed and operational
Docker Compose services	Running successfully
Docker bridge network	Configured and operational
Docker service-name resolution	Issues resolved by correcting service names in ***docker-compose.yaml***
API Gateway	Working correctly
Student registration	Working properly
Public courses and lesson selection	Verified working correctly
Examination materials	Working properly
Resource upload	Working properly
MySQL database	Operational with persistent storage
Frontend accessibility	Accessible through the deployed endpoint
Microservice communication	Verified working
GPT API integration	Tested and working properly
DeepSeek API integration	Tested and working properly
Gemini API integration	Tested and working properly
Third-party AI API integrations	Verified for the configured and tested integrations
End-to-end application workflows	Verified successfully

The status of historical issues should be updated only after new tests confirm the result.

7. Deployment Challenges and Resolutions

During the deployment of the application on AWS EC2 using Docker and Docker Compose, several configuration and integration issues were encountered. These issues were investigated and resolved to ensure that the frontend, backend microservices, database, and external AI services worked correctly in the deployed environment.

Table 3: Deployment challenges encountered and their corresponding resolutions during the AWS EC2 deployment of the LUMORA application.

Issue encountered	Resolution
Frontend API Gateway URL	Updated and verified the frontend gateway configuration to use the correct deployed endpoint instead of an outdated public IP address.
Docker DNS and service names	Corrected service names in docker-compose.yaml so containers could locate and communicate with one another through the Docker network.
JWT authentication and unauthorized API requests	Identified mismatches between JWT secrets and environment variables used by the frontend-related authentication configuration and backend services. Updated the configuration to ensure consistent token validation.
Model file paths during Dockerization	Corrected model file locations and Docker build configuration so the required model files were included in the container and could be accessed by the application.
Localhost and container networking	Replaced inappropriate localhost references in inter-service communication with the relevant Docker service names and container ports.
Gemini API key issues	Identified expired Gemini API keys and updated them with valid keys to restore external API communication.
NGINX configuration	Resolved reverse-proxy and request-routing configuration issues to ensure that frontend requests reached the API Gateway correctly.
Examination materials	Resolved the earlier API timeout issue and verified the examination materials workflow.
Resource upload restrictions	Addressed the earlier NGINX request-size issue and verified the resource-upload workflow.

Database and environment configuration	Corrected service configuration and environment variables required for database connectivity and application startup.
MySQL persistent storage	Configured and verified the separate persistent storage arrangement for the MySQL database.

8. Security and Reliability Considerations

The following considerations are important for publicly accessible, multi-service deployment.

Elastic IP and Security Groups: The Elastic IP provides a stable public address for accessing the application. Security Group rules control inbound traffic, so only the ports required for public access should be exposed.

Database protection: MySQL access should be restricted to authorized application services and administrative connections. The database should not be exposed directly to the public internet without a specific security requirement.

Environment variables and secrets: Database credentials, API keys, JWT signing secrets, and shared gateway secrets must be configured securely and excluded from source control. Weak default credentials should be avoided.

Frontend configuration: Vite environment variables embedded in the frontend bundle are visible to clients. Therefore, confidential credentials and private API keys must not be included in frontend configuration.

Persistent storage and backups: A separate persistent volume is used for MySQL database storage, keeping database files independent of the MySQL container lifecycle. This allows the database data to be retained when the container is restarted or recreated, provided the volume remains attached and is not deleted. Regular database backups should also be maintained separately to support recovery when required.

Resource Limitations: Running multiple Node.js and Python services on the same EC2 instance can increase memory and CPU usage. Resource utilization, available disk space, and container health should be monitored to maintain stable application performance.

HTTPS: Public application traffic should use HTTPS to protect data transmitted between clients and the deployed application. Certificate configuration, renewal, and domain settings should be maintained appropriately.